Portal do Patrocinador — Documentação Completa

Planeta Atlântida 2027 · Backend Spring Boot + Frontend React/Vite

Base desta documentação: análise do código do ficheiro `portal-integrado-final.zip` (backend, frontend, configurações e README). Os testes não foram executados e a aplicação não foi arrancada; tudo o que está aqui descreve o que o código faz, não o que foi validado em execução.

1. Visão geral

O Portal do Patrocinador é uma aplicação web onde a equipa RBS (organizadora do evento) e as marcas patrocinadoras trocam documentos, acompanham prazos de entrega, gerem cortesias e credenciamento, consultam resultados (Instagram e pesquisa de satisfação), geram contratos e conversam entre si.

Existem dois perfis de acesso no backend:

Perfil	Papel	O que vê
`ADMIN_RBS`	Equipa RBS	Todas as marcas, eventos, prazos, relatórios, configurações, histórico e base de conhecimento
`ADMIN_PATROCINADOR`	Utilizador de uma marca	Apenas a sua marca, e apenas as abas permitidas pela sua cota

O frontend acrescenta níveis internos para utilizadores de marca (administrador, marca e conteúdo, cortesias, credenciamento, só consulta) — ver secção 8.

2. Arquitetura (ponto mais importante)

O projeto contém **dois modelos de dados em paralelo**:

1. **Estado JSON do portal** — o frontend guarda a base de dados inteira (eventos, marcas, utilizadores, entregas, ficheiros, mensagens, etc.) num único documento JSON, enviado para `PUT /api/portal-state` e guardado na tabela `portal_front_state`. Há um estado para a RBS (`rbs`) e um por marca (`patrocinador-{id}`).

2. **API relacional (REST)** — o backend tem entidades JPA, serviços e controllers completos para patrocinadores, prazos, ficheiros, cortesias, credenciamento, contratos, alertas, mensagens, FAQ, Instagram, etc., com regras de acesso por cota.

O frontend atual usa apenas: login (`/api/auth/login` , `/api/auth/me`), estado do portal (`/api/portal-state`), pesquisa pública (`/api/public/pesquisa`), limpeza de respostas (`/api/portal-state/survey`) e criação de acessos de marca (`/api/portal-access/usuarios`). Todos os restantes endpoints existem e estão protegidos, mas **não são chamados pela interface**. Na prática, a lógica de negócio que o utilizador vê (prazos, cortesias, contratos, FAQ, histórico) corre no navegador sobre o JSON; a API relacional é uma base pronta para a evolução do sistema.

Consequências práticas: as regras de isolamento por cota e por perfil da API relacional **não protegem** os dados que vivem no estado JSON (ver secção 12).

```
Navegador (React) ──JWT──▶ Spring Boot :8080 ──▶ H2 (ficheiro) ou PostgreSQL
     |                          ├─▶ MongoDB: métricas Instagram (leitura)
     |                          ├─▶ MongoDB: ficheiros GridFS (opcional)
     |                          ├─▶ MongoDB: espelho/backup dos dados
(opcional)
     |                          ├─▶ Microsoft Graph: e-mail + SharePoint
(opcional)
     |                          ├─▶ SMTP (opcional) / e-mail simulado em log
     |                          └─▶ API Anthropic: assistente do FAQ
(opcional)
```

3. Stack tecnológica

Camada	Tecnologias
Backend	Java (pom: 17; README pede JDK 21), Spring Boot 3.3.5, Spring Web, Data JPA, Security, Validation, Mail, Lombok
Autenticação	JWT (jjwt 0.12.6, HS256), BCrypt, filtro `JwtAuthFilter`
Bases de dados	H2 em ficheiro (padrão), PostgreSQL (profile `postgres`), driver MongoDB sync
PDF	Apache PDFBox 3.0.3 (extração de texto do manual para a base de conhecimento)
Frontend	React 19, TypeScript 5.7, Vite 8, Tailwind CSS 4, Recharts, qrcode.react, class-variance-authority

Camada	Tecnologias
Ferramentas	Maven (sem `mvnw` no projeto), npm/pnpm (há `package-lock.json` e `pnpm-lock.yaml`)

O frontend nasceu no Figma Make (pasta `.figma/`, `AGENTS.md`, `CLAUDE.md`); o `vite.config.ts` inclui plugins específicos dessa plataforma.

4. Estrutura do repositório

```
portal-integrado-final/
├─ README.md                      instruções originais
├─ backend/
|  ├─ pom.xml
|  ├─ iniciar.ps1                 arranque no Windows (cria config local,
valida porta 8080)
|  ├─ config/application.example.yml   modelo da configuração local
|  ├─ config/application.yml      configuração local (NÃO deve ser
partilhada)
|  ├─ data/                       base H2 (portal.mv.db)
|  └─ src/main/java/br/com/rbs/portal/
|     ├─ config/       segurança, propriedades, seeders
|     ├─ domain/       entidades JPA e enums
|     ├─ dto/          objetos de pedido/resposta
|     ├─ repository/   Spring Data JPA
|     ├─ service/      regras de negócio
|     ├─ web/          controllers REST
|     ├─ security/     JWT
|     ├─ storage/      disco ou MongoDB GridFS
|     ├─ instagram/    métricas e alertas de menções
|     ├─ integration/  Graph, SharePoint, IA
|     └─ espelho/      backup H2 → MongoDB
└─ frontend/
   ├─ src/components/portal/PortalApp.tsx   aplicação inteira (~2600 linhas)
   ├─ src/lib/       backend.ts, audit.ts, contract.ts, faq.ts, survey.ts,
instagram.functions.ts
   ├─ public/branding/   imagens da identidade Planeta Atlântida 2027
   └─ .env.example   VITE_API_URL
```

5. Instalação e execução

Requisitos

JDK (17 conforme o `pom.xml`; o README indica 21), Maven 3.9+, Node.js 22+, npm. O README foi escrito para Windows PowerShell com VS Code.

Backend

```
cd backend
mvn spring-boot:run          # ou .\iniciar.ps1 no Windows
```

A API sobe em `http://localhost:8080`. O `iniciar.ps1` copia `config/application.example.yml` para `config/application.yml` se não existir, verifica se a porta 8080 está livre e executa `mvn spring-boot:run`. A base H2 é criada em `backend/data/`.

Frontend

```
cd frontend
Copy-Item .env.example .env     # VITE_API_URL=http://localhost:8080
npm install
npm run dev                     # http://localhost:8081
npm run build
```

Primeiro login

Com `SEED_DEMO=true` (valor padrão) e base vazia, o backend cria `admin@rbs.com.br` (senha inicial documentada no README) e cinco marcas de demonstração (Solar, Banco do Sul, Nativa, Cooperativa Horizonte, Universo Tech). A conta RBS deve entrar primeiro, porque é ela que inicializa o estado JSON no H2; só depois as marcas conseguem entrar. **Desative `SEED_DEMO` e troque todas as senhas antes de qualquer uso real.**

PostgreSQL

```
$env:DB_URL='jdbc:postgresql://localhost:5432/portal'
$env:DB_USER='portal'; $env:DB_PASSWORD='...'
$env:SPRING_PROFILES_ACTIVE='postgres'
mvn spring-boot:run
```

6. Configuração

A ordem de precedência é: variáveis de ambiente → `backend/config/application.yml` → `src/main/resources/application.yml`. Principais propriedades (prefixo `portal.`):

Variável de ambiente	Propriedade	Padrão	Função
`JWT_SECRET`	`jwt.secret`	valor de exemplo	Segredo do JWT (mín. 32 caracteres; a aplicação não arranca com menos)
—	`jwt.expiration-minutes`	480	Validade do token (8 h)
`CORS_ORIGINS`	`cors-origins`	localhost 5173, 8081, 8443	Origens permitidas
`FRONTEND_URL`	`frontend-url`	`http://localhost:5173`	Usada nos links dos e-mails
`EVENTO_NOME` / `EVENTO_DATA`	`evento.*`	Planeta Atlântida / 2027-01-29	Evento padrão e base dos prazos
`EMPRESA_NOME` / `EMPRESA_CNPJ`	`empresa.*`	vazio	Preenchem o contrato gerado
`OPEN_STAFF_URL`	`open-staff-url`	exemplo	Link do sistema de credenciamento
`STORAGE_DIR`	`storage-dir`	`./uploads`	Pasta dos ficheiros (modo disco)
`STORAGE_TIPO`	`storage.tipo`	`disco`	`disco` ou `mongo` (GridFS)
`ESPELHO_ATIVO`	`espelho.ativo`	false	Backup H2 → MongoDB
`ALERTAS_CRON`	`alertas.cron`	`0 0 8 * * *`	Job diário de prazos (`-` desliga)
`MONGODB_URI`	`instagram.mongo-uri`	vazio	Liga as métricas do Instagram
`INSTAGRAM_CRON`	`instagram.cron`	a cada 3 h	Procura menções às marcas
`INSTAGRAM_EVENTO_USERNAME`	`instagram.evento-username`	`planetaatlantida`	Perfil oficial do evento
`GRAPH_ENABLED` e `GRAPH_*`	`graph.*`	false	Microsoft 365 (e-mail e SharePoint)
`SPRING_MAIL_*`	`spring.mail.*`	—	SMTP

Variável de ambiente	Propriedade	Padrão	Função
`IA_ENABLED`, `ANTHROPIC_API_KEY`, `IA_MODEL`	`ia.*`	false / `claude-sonnet-5-5`	Assistente do FAQ
`SEED_DEMO`	`seed-demo`	true	Contas e marcas de demonstração

Uploads: limite multipart de 50 MB por ficheiro (55 MB por pedido). O `GET /api/config/status` (só RBS) mostra o que está ligado, sem expor segredos.

7. Modelo de domínio (backend)

Entidade	Descrição
`Evento`	Edição do evento (nome, início, fim, local). Os prazos contam a partir do início
`Patrocinador`	Marca: nome, cota, @Instagram, evento e `frontendId` (liga ao id usado no estado JSON)
`Usuario`	E-mail, hash BCrypt, perfil, marca (uma só) e flag `ativo`
`Arquivo`	Metadados do ficheiro (aba, categoria, nome original, UUID armazenado, quem enviou)
`ItemConteudo`	Textos/links por aba: material de mídia, link de post, signatário
`Prazo`	Entrega com data-limite; o estado é calculado, não gravado
`Cortesia` / `Credenciamento`	Quantidades por setor (Arena, Camarote, Premium, Lounge) e tipo (passaporte, sexta, sábado)
`PerguntaPesquisa`	Até 3 perguntas personalizadas por marca e respetivo resultado
`ModeloContrato` / `ContratoGerado`	Modelos com variáveis `{{...}}` e contratos emitidos (texto congelado, versão, visibilidade ao cliente)
`Alerta` / `AlertaLeitura`	Notificações com chave única; leitura individual
`Mensagem`	Conversa marca ↔ RBS (uma por marca)

Entidade	Descrição
`Historico`	Quem fez o quê e quando, por marca
`FaqItem` / `ConhecimentoDoc` / `FaqPergunta`	FAQ, documentos do manual e registo das perguntas feitas
`TokenSenha`	Token de recuperação de senha (só o hash SHA-256)
`PortalState`	JSON do estado do frontend (`portal_front_state`)

Cotas e abas

Sete abas: Perfil, Aplicações de marca, Mídia e conteúdo, Cortesias, Credenciamento, Pesquisa, Contrato.

Cota	Abas no backend
Master	Todas (7)
Subcota G	Perfil, Mídia, Cortesias, Pesquisa, Contrato
Subcota	Perfil, Mídia, Pesquisa

O código assinala que esta distribuição é uma assunção do MVP (o documento de projeto não detalha as abas por cota). No frontend, a aba Pesquisa só aparece para a cota Master e o credenciamento completo só existe na Master — há, portanto, uma diferença entre backend e frontend a validar com a RBS.

Categorias de ficheiro e quem pode enviar

Categoria	Aba	Marca pode enviar?
Plano comercial	Perfil	Não
Compra de ingressos	Perfil	Sim
MIV, Logo	Aplicações	Sim
Fotos de comprovação	Aplicações	Não
Materiais do cliente	Mídia	Sim
Relatórios de mídia	Mídia	Não

Categoria	Aba	Marca pode enviar?
Resultados da pesquisa	Pesquisa	Não
Contrato assinado	Contrato	Sim

Extensões bloqueadas no upload: `exe, bat, cmd, com, msi, sh, jar, js, vbs, ps1, scr, dll`. As marcas só podem apagar o que elas próprias enviaram.

8. Regras de negócio

Prazos

Prazos padrão (dias antes do evento): ingressos 90, signatário 100, logo 75, MIV 75, material de mídia 45, perguntas da pesquisa 60, destinar cortesias 30. São gerados ao criar a marca ou mudar a cota (idempotente), apenas para abas que a cota inclui, e reagendados quando a data do evento muda.

Estado calculado: **Entregue** (a entrega existe no portal) · **Atrasado** (data já passou) · **Urgente** (faltam 7 dias ou menos) · **No prazo**. As entregas são detetadas automaticamente (por exemplo, existir um ficheiro da categoria certa ou 3 perguntas preenchidas); só os prazos *personalizados* são marcados à mão. O calendário pode ser exportado em `.ics` (`GET /api/calendario.ics`, com lembrete 3 dias antes).

Alertas

Um job diário às 08:00 (fuso de São Paulo) cria alertas nos marcos D-7, D-3, D-1, dia do vencimento e uma vez por semana de atraso, e envia e-mail aos utilizadores ativos da marca. A `chave` única garante um alerta por marco. Uploads, contratos publicados e mensagens também geram alertas. Atrasos são também notificados à RBS.

Contratos (backend)

A RBS mantém modelos com variáveis como `{{patrocinador.nome}}`, `{{signatario.nome}}`, `{{evento.data}}`, `{{cortesias.total}}`, `{{empresa.cnpj}}`. Variáveis sem valor ficam visíveis no texto e são devolvidas como *pendentes*. O contrato emitido é congelado. A marca só vê contratos publicados. «Excluir» um modelo apenas o arquiva.

Nota: o frontend tem o seu próprio gerador de contratos (`contract.ts`) com outras variáveis (`{{marca}}`, `{{cota}}`, `{{valor_investimento}}` …), independente do backend.

Assistente de dúvidas (FAQ)

Pesquisa tf·idf em português (sem acentos, radicais de 5 letras) sobre o FAQ e os documentos do manual. Com `IA_ENABLED=true` e chave, a resposta é redigida por um modelo da Anthropic usando apenas os trechos encontrados (o prompt trata o contexto como dados, não instruções); sem IA devolve o melhor item do FAQ. Perguntas sem resposta são registadas para a RBS completar a base. Limite de 30 perguntas por hora por utilizador. O PDF do manual é carregado em `POST /api/conhecimento/arquivo`.

Instagram

Lê, em modo só de leitura, dois conjuntos no MongoDB (perfis diários e posts). Calcula seguidores, variação 7/30 dias, crescimento, engajamento, top posts, desempenho por tipo e dia da semana e melhor horário. Um job a cada 3 horas procura posts do perfil do evento que citam `@marca` e gera um alerta por post. Cache de 5 minutos. Sem `MONGODB_URI` a funcionalidade fica desligada. No frontend, `instagram.functions.ts` só devolve dados reais para a conta de demonstração `teste`; para as restantes devolve erro de «ligação não configurada».

9. Segurança implementada

- Sessão stateless com Bearer JWT; CSRF desativado (não usa cookies).
- O filtro recarrega o utilizador a cada pedido: desativar um e-mail ou mudar o perfil tem efeito imediato.
- Regras globais: `/api/auth/login`, `/esqueci-senha`, `/redefinir-senha` e `/api/public/**` são públicas; `/api/**` exige autenticação; o resto é negado.
- Autorização por método (`@PreAuthorize`) e centralizada em `AcessoService` (acesso à marca e à aba pela cota).
- Login: mensagem única para credenciais erradas; bloqueio de 15 min após 5 falhas por IP+e-mail.
- Recuperação de senha: token aleatório de 256 bits, guardado só como hash, válido 60 min, uso único, resposta sempre igual exista ou não o e-mail, limite de 5 pedidos por hora.
- Força da senha (reset/troca): mínimo 8 caracteres com letra e número.
- Downloads com `Content-Disposition: attachment` e `nosniff`; proteção contra *path traversal* no armazenamento em disco.
- Erros 401/403/404 distintos para marca sem acesso.

10. Referência da API

Prefixo `/api`. «RBS» = apenas `ADMIN_RBS`; «Marca*» = a marca só acede aos seus dados e às abas da sua cota.

Área	Endpoints	Acesso
Autenticação	`POST /auth/login`, `GET /auth/me`, `POST /auth/esqueci-senha`, `POST /auth/redefinir-senha`, `PUT /auth/senha`	público (exceto me/senha)
Estado do portal	`GET/PUT /portal-state`, `DELETE /portal-state/survey`	autenticado (âmbito por perfil)
Pesquisa pública	`GET/POST /public/pesquisa`	público
Acesso de marcas	`POST /portal-access/usuarios`	RBS
Eventos	`GET /eventos`; `POST`, `PUT /{id}`, `DELETE /{id}`	GET: todos; resto RBS
Patrocinadores	`GET /patrocinadores`, `GET /{id}`, `GET /{id}/painel`, `GET /{id}/relatorio`; `POST`, `PUT /{id}`	Marca*; escrita RBS
Utilizadores	`GET/POST /patrocinadores/{pid}/usuarios`, `PATCH /{uid}/ativo`, `PUT /{uid}/senha`	Marca* (escrita RBS)
Ficheiros	`GET/POST /patrocinadores/{pid}/arquivos`, `GET /arquivos/{id}/download`, `DELETE /arquivos/{id}`	Marca*
Itens	`GET/POST /patrocinadores/{pid}/itens`, `DELETE /itens/{id}`	Marca*
Prazos	`GET /prazos`, `GET/POST /patrocinadores/{pid}/prazos`, `PUT/DELETE /prazos/{id}`, `PATCH /prazos/{id}/concluido`, `POST .../prazos/padrao`, `GET /calendario.ics`	leitura Marca*; escrita RBS
Cortesias	`GET/POST /patrocinadores/{pid}/cortesias`, `PUT/DELETE /{id}`, `PATCH /{id}/destinar`	Marca* (definir: RBS; destinar: ambos)

Área	Endpoints	Acesso
Credenciamento	`GET/POST /patrocinadores/{pid}/credenciamento`, `DELETE /{id}`	leitura Marca*; escrita RBS
Pesquisa	`GET /patrocinadores/{pid}/pesquisa`, `PUT /{ordem}/pergunta`, `PUT /{ordem}/resultado`	pergunta: Marca*; resultado: RBS
Contratos	`GET/POST/PUT/DELETE /contratos/modelos`, `GET /contratos/variaveis`, `GET/POST /patrocinadores/{pid}/contratos`, `POST .../contratos/previa`, `PATCH/DELETE /contratos/{id}`	RBS (leitura dos contratos publicados: Marca*)
Alertas	`GET /alertas`, `POST /alertas/{id}/lida`, `POST /alertas/lidas`, `POST /alertas/executar`	executar: RBS
Mensagens	`GET/POST /patrocinadores/{pid}/mensagens`, `GET /mensagens/nao-lidas`, `GET /mensagens/caixa`	caixa: RBS
Histórico	`GET /patrocinadores/{pid}/historico`	Marca*
Painel	`GET /painel`	RBS
FAQ	`GET /faq`, `POST /faq/perguntar`; RBS: `/faq/todos`, `POST/PUT/DELETE /faq`, `/faq/perguntas`, `/conhecimento` (+ `/arquivo`)	misto
Instagram	`GET /instagram/evento`, `GET /patrocinadores/{pid}/instagram`; RBS: `POST /instagram/testar`, `/instagram/sincronizar`	misto
Meta/Config	`GET /meta`, `GET /config/status`	meta: autenticado; config: RBS

11. Frontend

É uma aplicação de página única (`PortalApp.tsx`) sem router: a navegação é um estado `page` . O login pré-preenche o e-mail `admin@rbs.com.br` .

Páginas

Visão geral, Evento e clientes (RBS), Perfil do cliente, Aplicações de marca, Mídia e conteúdo (com Arquivos solicitados e Prazos), Cortesia e Credenciamento, Pesquisa da ativação, Contrato, Dashboard (analytics), Relatório, Contato, Dúvidas frequentes e Histórico (só RBS).

Níveis de acesso das marcas (aplicados só no frontend)

Nível	Páginas
Administrador da marca	todas as de cliente
Marca e conteúdo	visão geral, marca, mídia, pedidos, entregas, calendário, dashboard, relatório, contato, FAQ
Cortesias e convidados	visão geral, cortesias, dashboard, relatório, contato, FAQ
Credenciamento	como o anterior, mais credenciamento
Somente consulta	visão geral, dashboard, relatório, contato, FAQ

Ciclo de vida do estado

1. No arranque lê o token de `localStorage` (`planeta-portal-api-token`) e chama `/auth/me` e `/portal-state`.
2. Se o estado remoto estiver vazio e o utilizador for RBS, semeia o estado a partir dos dados de demonstração embutidos no código (também espelhados em `localStorage`, chave `planeta-portal-mvp-v1`).
3. Cada alteração passa por `update()`, que calcula um diff e acrescenta entradas ao histórico de auditoria (limite de 2000).
4. O estado é gravado no servidor com *debounce* de 350 ms.

Outros pormenores

- Upload no frontend: máximo de 750 KB (imagens são redimensionadas); o conteúdo fica dentro do estado JSON.
- Exportação Excel `.xlsx` gerada no navegador (`excelExport.ts`, sem dependências) com cabeçalhos, filtros e larguras de coluna.
- Relatórios gerais podem ser guardados no estado (formato CSV interno) e descarregados como Excel.

- Pesquisa de satisfação: três notas (atendimento, organização, espaço) mais comentário; a média é calculada; há geração de respostas simuladas para testes.
- Identidade visual PA27 em `public/branding/`; os `.asset.json` em `src/assets/` são só metadados — os ficheiros originais (logo, capa, palco, manual em PDF) **não vêm no zip**.

12. Limitações e riscos identificados

Ordenados por gravidade. São observações da análise do código, não resultados de testes de intrusão.

Alta

1. **Credencial real no pacote.** `backend/config/application.yml` contém a URI do MongoDB Atlas com utilizador e senha. O ficheiro devia estar no `.gitignore` e não ser enviado. Recomenda-se **rodar essa senha** e remover o ficheiro de qualquer partilha. Pela mesma razão, `backend/data/portal.mv.db` (base com dados reais ou de teste) não deve circular.
2. **Isolamento entre marcas no estado JSON.** Na primeira visita, se a marca ainda não tem estado próprio, `GET /portal-state` devolve o estado completo da RBS (com as senhas removidas), incluindo dados de outras marcas, mensagens e ficheiros. Depois disso, cada marca escreve o seu estado sem validação do servidor.
3. **Senhas no estado JSON.** O formulário de utilizadores guarda a senha no objeto `users` do estado da RBS (texto simples), e o espelho MongoDB copia essa tabela. A autenticação real usa BCrypt, mas a cópia em texto simples não devia existir.
4. **Segredo JWT por omissão.** Sem `JWT_SECRET`, o segredo é um valor de exemplo público no repositório; qualquer pessoa poderia forjar tokens.
5. **Pesquisa pública sem proteção.** `POST /api/public/pesquisa` é anónimo, sem limite de pedidos nem validação do evento/marca, e escreve no estado da RBS (possível spam e crescimento do JSON).

Média

6. **Dois modelos de dados.** As regras do backend (prazos, alertas, contratos, cotas) não alimentam a interface; contratos e abas por cota divergem entre `contract.ts`/frontend e o backend. O contrato enviado pela marca fica no estado dela e **não aparece** na conta RBS (o README avisa que não serve como prova de receção).
7. **Níveis de acesso só no frontend.** Qualquer utilizador de marca autenticado pode alterar o seu estado diretamente pela API.

8. `ddl-auto: update` em produção e **H2 em ficheiro** são adequados a MVP; sem migrações (Flyway/Liquibase). H2 com `AUTO_SERVER` não é indicado para produção.
9. **Limitador de tentativas em memória**: não funciona com várias instâncias e perde-se ao reiniciar; depende de `getRemoteAddr` (atrás de proxy verá o IP do proxy).
10. **Token em `localStorage`**: exposto a qualquer XSS.
11. **Estado único**: cada gravação envia o JSON inteiro; ficheiros em base64 dentro do JSON não escalam.

Baixa / informação

12. Versão de Java: `pom.xml` define 17, README pede 21. Confirmar qual é a oficial.
13. Integrações **Graph/SharePoint** e **IA** estão marcadas no código como não testadas contra serviços reais; SharePoint só sincroniza ficheiros até 4 MB.
14. Não há Dockerfile, CI, `mvnw` nem documentação OpenAPI. A pasta `.github/modernize` contém apenas *hooks* de uma ferramenta de atualização de Java.
15. As marcas de demonstração usam senhas previsíveis (listadas no README).
16. O frontend não corrige acentuação/segurança de `README` antigo: o texto «Correção de login das marcas (403)» refere comportamento já incorporado em `PortalAccessController`.

13. Testes

Existem testes JUnit/Spring no backend (`mvn test`, perfil `application-test.yml` com H2 em memória e jobs desligados):

Teste	Cobre
`SegurancaIsolamentoTest`	isolamento entre marcas e perfis
`EventosMensagensTest`	eventos e conversas
`SenhaRecuperacaoTest`	recuperação e troca de senha
`RegrasPurasTest`, `PainelEEventoRegrasTest`, `AlertaServiceTest`	prazos, marcos de alerta, painel, estado de eventos
`InstagramMetricasTest`, `InstagramDesligadoTest`	cálculo de métricas e modo desligado
`ArmazenamentoDiscoTest`	armazenamento em disco

O frontend não tem testes automáticos; a verificação é `npm run build`.

14. Operação

- **Backups:** copiar `backend/data/` (H2) ou ativar o espelho MongoDB (`espelho.ativo`), que sincroniza a cada 2 minutos para coleções `sistema_<tabela>` e restaura automaticamente se a base local arrancar vazia.
- **Ficheiros:** em `STORAGE_DIR` (disco) ou GridFS (`portal_arquivos`). Usar o mesmo modo entre reinícios, ou os ficheiros antigos deixam de ser encontrados.
- **E-mail:** ordem de preferência Graph → SMTP → simulado (apenas log, incluindo links de recuperação).
- **Logs úteis:** «Verificação de prazos», «Menções do Instagram», «[E-MAIL SIMULADO]».
- **Antes de produção:** definir `JWT_SECRET`, `SEED_DEMO=false`, `CORS_ORIGINS`, `FRONTEND_URL`, `EMPRESA_*`, PostgreSQL, HTTPS (proxy reverso), rodar credenciais expostas e rever as questões da secção 12.

15. Glossário

RBS — organizadora do evento. **Cota** — nível de patrocínio (Master, Subcota G, Subcota). **MIV** — manual de identidade visual. **Open Staff** — sistema externo de credenciamento. **Espelho** — cópia dos dados do H2 no MongoDB. **Estado do portal** — documento JSON com os dados do frontend. **Marco** — dia de aviso de um prazo (D-7, D-3, D-1, D0, atraso semanal).